POSTGRESQL · PL/PGSQL

Function or procedure?

Which one to write, and the errors you get when you pick the wrong one.

SAME LANGUAGE, DIFFERENT JOBS

Two kinds of routine

	FUNCTION	PROCEDURE
Returns a value	Yes	No, uses OUT params
Called with	SELECT	CALL
Inside a query	Yes	No
COMMIT / ROLLBACK	No	Yes

Procedures arrived in PostgreSQL 11 (2018).

FUNCTION

Returns a value

```
CREATE FUNCTION order_total(p_cust int)
RETURNS numeric
LANGUAGE plpgsql AS $$
BEGIN
  RETURN (SELECT sum(amount) FROM orders
          WHERE customer_id = p_cust);
END $$;

SELECT order_total(101);
```

Use it anywhere an expression fits: in a SELECT list, WHERE or ORDER BY.

PROCEDURE

Runs a job, commits as it goes

```
CREATE PROCEDURE archive_orders(p_before date)
LANGUAGE plpgsql AS $$
BEGIN
  LOOP
    WITH moved AS (
      DELETE FROM orders
      WHERE order_id IN (
        SELECT order_id FROM orders
        WHERE order_date < p_before
        LIMIT 1000)
      RETURNING *)
    INSERT INTO orders_archive
    SELECT * FROM moved;
    EXIT WHEN NOT FOUND;
    COMMIT;  -- every 1,000 rows
  END LOOP;
END $$;
```

RUN IT WITH CALL

Small batches, each one committed

```
CALL archive_orders('2025-01-01');
```

On a 10,000-row test table:

check	count
rows in orders_archive	4126
old rows left in orders	0

Each COMMIT ends the transaction, so locks are released between batches and a failure keeps the batches already done.

MISTAKE 1

COMMIT inside a function

```
CREATE FUNCTION archive_old(p_before date)
RETURNS void
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM orders
  WHERE order_date < p_before;
  COMMIT;
END $$;

SELECT archive_old('2025-06-01');
```

```
ERROR:  invalid transaction termination
```

Functions run inside the caller's transaction. Need COMMIT? Write a procedure.

MISTAKE 2

Calling it the wrong way

```
SELECT archive_orders('2025-06-01');
```

```
ERROR:  archive_orders(unknown) is a procedure
HINT:  To call a procedure, use CALL.
```

```
CALL order_total(101);
```

```
ERROR:  order_total(integer) is not a procedure
HINT:  To call a function, use SELECT.
```

PostgreSQL tells you which one you need. Read the HINT line.

MISTAKE 3

CALL inside a BEGIN block

```
BEGIN;
CALL archive_orders('2025-06-01');
```

```
ERROR:  invalid transaction termination
```

A procedure can only COMMIT when its CALL is not inside an explicit transaction block.

Run it on its own, in autocommit mode.

RECAP · SCREENSHOT THIS

Pick the right one

- ✔ **Function:** returns a value, used inside SELECT
- ✔ **Procedure:** no RETURNS, run with **CALL**
- ✔ Only procedures can **COMMIT / ROLLBACK**
- ✔ Not inside **BEGIN** if the procedure commits
- ✔ Batch jobs and archiving: **procedure**

BEFORE YOU GO

Save this for your next batch job.

All code tested on PostgreSQL 16.
Procedures need PostgreSQL 11 or later.